

User Manual & Technical Architecture

Guide: LLM Load Testing Tool

This document provides a structured table of contents followed by the operational user manual and technical guide for the **LLM Load Testing Tool**, updated to reflect the provided user interface and dashboard metrics.

Document Index

1. System Overview & Architecture

- Key Technical Highlights
- Core Architectural Pipeline

2. Quick Start & Installation

- Prerequisites
- Setup Instructions

3. UI Configuration Parameters Guide

- Input Field Reference
- System Behavior Configuration

4. Dashboard Metrics & UI Analysis

1. System Overview & Architecture

The **LLM Load Testing Tool** is a self-contained web application built to stress-test OpenAI-compatible Large Language Model (LLM) inference endpoints (e.g., vLLM, TGI, llama.cpp server, Ollama, TensorRT-LLM). It measures throughput, latency, and reliability under varying concurrent user counts and prompt token context lengths.

Key Technical Highlights

- **Flask Web Server (app.py):** Serves the UI, validates form input, manages background thread lifecycles, and exposes status polling endpoints.
- **Load Engine (engine.py):** Fires concurrent HTTP requests via Python thread pools, tracks usage statistics, collects GPU telemetry using nvidia-smi, generates charts with Matplotlib, and writes PDF reports using ReportLab.
- **Thread Safety:** Protects global state with threading.Lock() structures and uses threading.Event() for clean cancellation.
- **Zero External DB:** Operates entirely in memory during active runs and saves output files to runs/<run_id>/ (results.json, chart PNGs, and report.pdf).

2. Quick Start & Installation

Prerequisites

- **Python Environment:** Python 3.9+ installed.
- **LLM Inference Server:** An active endpoint serving OpenAI-compatible chat completion requests (e.g., <http://localhost:8000/v1/chat/completions>).
- **GPU Monitoring (Optional):** NVIDIA GPU environment running `nvidia-smi` for VRAM and power metrics.

Setup Instructions

1. **Clone the repository:**

```
git clone https://github.com/Alexjmathew7/LLM_TESTING.git  
cd LLM_TESTING
```
2. **Set up a virtual environment:**

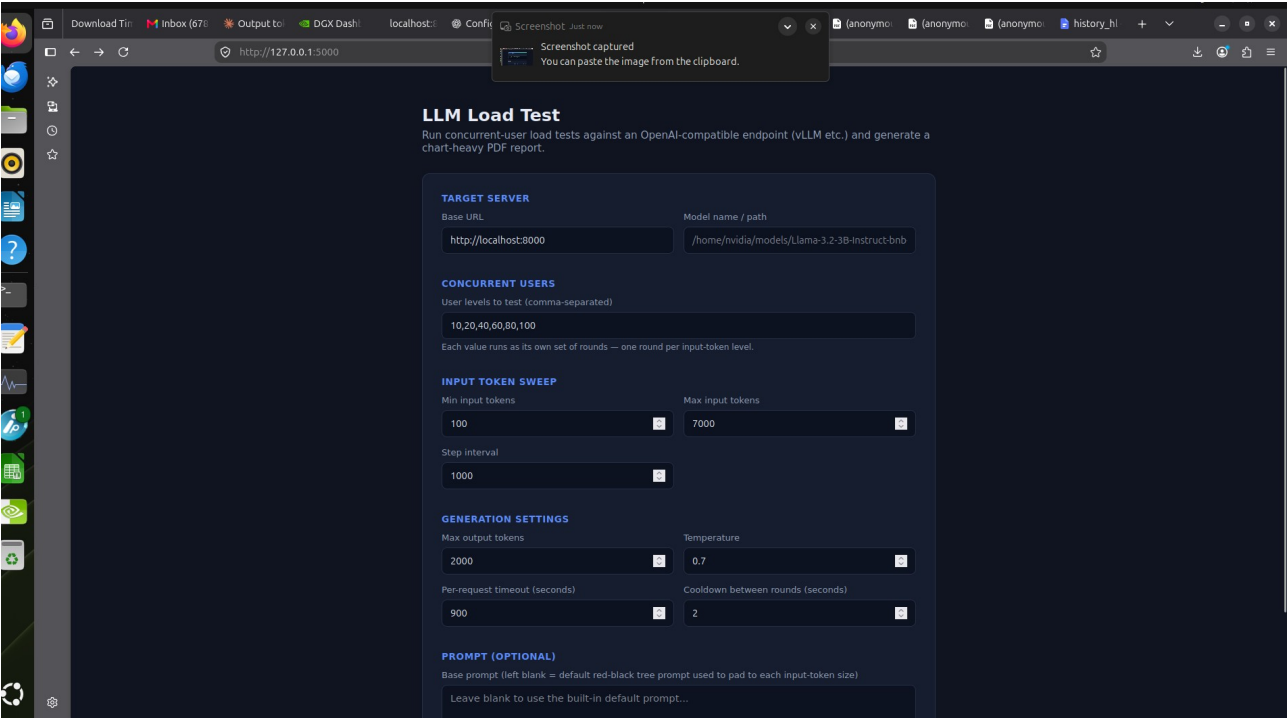
```
python3 -m venv venv  
source venv/bin/activate
```
3. **Install dependencies:**

```
pip install flask matplotlib reportlab
```
4. **Launch the web application:**

```
python app.py
```
5. **Open the Web UI:**
Navigate to <http://localhost:5000> in your web browser.

3. UI Configuration Parameters Guide

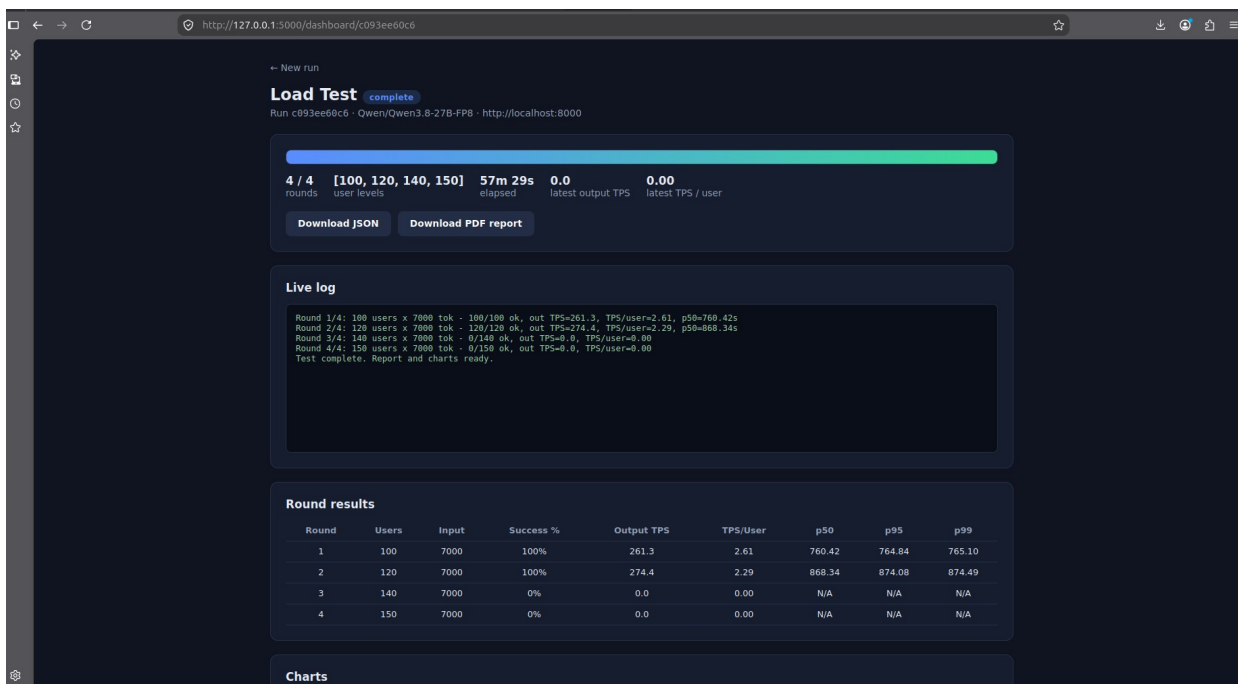
The launch screen lets you configure target endpoints, user concurrency, input sweeps, and output settings.



Parameter Name	Function	Typical / Example Value
Base URL	Target HTTP base address for the model server.	http://localhost:8000
Model name / path	Model path identifier expected by the server.	Qwen/Qwen3.8-27B-FP8 or local directory path
User levels	Comma-separated list of concurrent request workers per round.	100, 120, 140, 150
Min / Max input tokens	Defines the bounds for generated prompt size sweeps.	100 to 7000
Step interval	Incremental token size increase per test step.	1000

Parameter Name	Function	Typical / Example Value
Max output tokens	Maximum completion tokens generated per call.	2000
Temperature	Sampling variance parameter.	0.7
Per-request timeout	Maximum allowed response duration before failing.	900
Cooldown between rounds	Delay (in seconds) between rounds to clear GPU VRAM.	2

4. Dashboard Metrics & UI Analysis



During or after a run, the UI presents metrics across four main sections:

Dashboard Elements

- **Progress Header Cards:** Displays total completed rounds, user level array, wall-clock time elapsed, and latest measured throughput numbers.
- **Live Terminal Log:** Text window streaming progress lines and errors from active testing rounds.
- **Download Actions:** Quick links to retrieve results.json raw data and the compiled report.pdf.

Metric Definitions (Round Results Table)

- **Users:** Number of simultaneous workers sending requests.
- **Input:** Prompt context length (in tokens).
- **Success %:** Ratio of requests completed successfully without timeouts or errors.
- **Output TPS (Tokens Per Second):** Aggregate system generation speed across all active threads

$$\text{Output TPS} = \frac{\sum \text{Completion Tokens}}{\text{Round Wall Clock Time}}$$

- **TPS/User:** Perceived individual user generation speed:

$$\text{TPS/User} = \frac{\text{Output TPS}}{\text{Users}}$$

Prevent Request Contamination: Always keep Cooldown between rounds above 0 (recommended: 2 to 5 seconds) to let backend memory clear between rounds.

- Adjust Timeout for High Concurrency: Set Per-request timeout higher (e.g., 900 seconds) when testing large batch loads so requests waiting in the server queue are not cancelled prematurely.
- Diagnosing 0% Success Rates: If rounds fail entirely, check server logs for Out-Of-Memory (OOM) errors or context window limit violations.

The LLM Load Testing Tool provides an end-to-end framework for evaluating LLM inference engines. By combining lightweight concurrency, thread-safe background execution, real-time feedback, and automated report generation, it gives engineers the insights needed to optimize model deployments, size infrastructure accurately, and ensure high reliability under real-world traffic patterns.